파이썬을 이용한 데이터 과학
(Data Science using Python)

6주차 강의

넘파이 활용
- 다차원 배열 구조 변경



담당교수: 박명훈 (parc.seoultech@seoultech.ac.kr)

•본 과제(결과물) 는 교육부와 한국연구재단의 재원으로 지원을 받아 수행된 첨단분야 혁신융합 대학사업의 연구결과입니다 .

배열 결합

- **np.stack**: 배열을 쌓아서 구성하는 방법.

```
x = np.array([1,2,3])
y = np.array([4,5,6])
```

```
np.stack([x,y])
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
np.stack([x,y], axis = 0)
```

: 수직으로 쌓기

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
z = np.array([7,8,9,10])
try:
    np.stack([x,z],axis=0)
except Exception as e :
    print(e)
```

```
all input arrays must have the same shape
```

: 수평으로 쌓기

```
np.stack([x,y], axis = 1)
```

```
array([[1, 4],
       [2, 5],
       [3, 6]])
```

: 임의의 축 지정

(1차원의 경우 열을 기준)

```
np.stack([x,y], axis = -1)
```

```
array([[1, 4],
       [2, 5],
       [3, 6]])
```

- **축 지정 없이 배열을 쌓기**

```
x = np.array([1,2,3])
y = np.array([4,5,6])
```

```
a = np.vstack([x,y])
```

```
a
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
np.vstack([a,a])
```

```
array([[1, 2, 3],
       [4, 5, 6],
       [1, 2, 3],
       [4, 5, 6]])
```

```
np.vstack([[a,a]])   # 3차원 배열로 쌓임.
```

```
array([[[1, 2, 3],
        [4, 5, 6]],

       [[1, 2, 3],
        [4, 5, 6]]])
```

```
b= np.hstack([x,y])
```

```
b
```

```
array([1, 2, 3, 4, 5, 6])
```

```
np.hstack([b,b])
```

```
array([1, 2, 3, 4, 5, 6, 1, 2, 3, 4, 5, 6])
```

```
np.hstack([[b,b]])
```

```
array([[1, 2, 3, 4, 5, 6],
       [1, 2, 3, 4, 5, 6]])
```

- **dstack:** 축을 확대하기 (depth-wise stacking), axis= 2 (세번째 축 기준)
  $(m, n) \rightarrow (m, n, k)$, k는 쌓은 배열의 개수

```
a
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

shape = (2,3)

```
np.dstack([a,a])
```

```
array([[[1, 1],
        [2, 2],
        [3, 3]],

       [[4, 4],
        [5, 5],
        [6, 6]]])
```

shape = (2,3,2)

```
x = np.arange(1,4).reshape(3,-1)
x
```

```
array([[1],
       [2],
       [3]])
```

```
y = np.arange(10,13).reshape(3,-1)
y
```

```
array([[10],
       [11],
       [12]])
```

```
np.dstack([x,y])
```

```
array([[[ 1, 10]],

       [[ 2, 11]],

       [[ 3, 12]]])
```

```
np.dstack([x,y]).shape
```

```
(3, 1, 2)
```

```
x = np.arange(1,4)
x
```

```
array([1, 2, 3])
```

```
y = np.arange(10,13)
y
```

```
array([10, 11, 12])
```

```
np.dstack([x,y])
```

```
array([[[ 1, 10],
        [ 2, 11],
        [ 3, 12]]])
```

```
np.dstack([x,y]).shape
```

```
(1, 3, 2)
```

축	의미
`axis=0`	행 방향 쌓기 (`vstack`)
`axis=1`	열 방향 쌓기 (`hstack`)
`axis=2`	**깊이(depth) 방향 쌓기** (`dstack`) ← 이 경우 3D 배열 생성

함수	주 결합 축	입력 형태	차원 변화	설명
`np.concatenate`	지정 축 (`axis`)	튜플/리스트	유지 또는 그대로	가장 일반적인 결합
`np.stack`	새 축 생성 (`axis`)	튜플/리스트	⬆️ 1 증가	동일 shape 필요, 새 축 추가
`np.hstack`	수평 결합 (`axis=1`)	튜플/리스트	유지 또는 2D로	컬럼 방향으로 붙임
`np.vstack`	수직 결합 (`axis=0`)	튜플/리스트	유지 또는 2D로	행 방향으로 붙임
`np.dstack`	깊이 결합 (`axis=2`)	튜플/리스트	⬆️ 1 증가	2D → 3D 쌓기

배열 변경

- reshape: 배열의 형상 변경

```
try:
    x2 = x.reshape(4,5)
except Exception as e:
    print(e)
```

```
cannot reshape array of size 16 into shape (4,5)
```

```
x = np.arange(16)
y = x.reshape(4,4)
np.may_share_memory(x,y)
```

```
True
```

- resize: 배열의 크기를 늘리거나 줄이기

```
a = np.arange(4)
```

```
b=np.resize(a,8)
b
```

```
array([0, 1, 2, 3, 0, 1, 2, 3])
```

```
np.append(a,a)
```

```
array([0, 1, 2, 3, 0, 1, 2, 3])
```

```
a.resize((4,4))
a
```

```
array([[0, 1, 2, 3],
       [0, 0, 0, 0],
       [0, 0, 0, 0],
       [0, 0, 0, 0]])
```

- resize: 배열의 크기를 늘리거나 줄이기
  - resize 메소드를 시용해 배욜의 크기를 변경할 때, refcheck=False 를 이용하여 기존 배열의 레퍼런스를 참조하지 않도록 지정해야 원본 배열 변경 가능

```
a = np.array([1, 2, 3])
b = a  # b는 a를 참조
np.may_share_memory(a,b)
```

```
True
```

```
try:
    a.resize((5,))
except Exception as e:
    print(e)
```

```
cannot resize an array that references or is referenced
by another array in this way.
Use the np.resize function or refcheck=False
```

```
a.resize((5,), refcheck=False)
```

```
b
```

```
array([1, 2, 3, 0, 0])
```

```
d = np.arange(16)
d
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15])
```

```
d.resize(3,3)
d
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[31], line 1
----> 1 d.resize(3,3)
      2 d

ValueError: cannot resize an array that references or is referenced
by another array in this way.
Use the np.resize function or refcheck=False
```

- 출력 시스템에 의해 자동 참조가 되기 때문에, 에러 메세지가 발생.

```
d = np.arange(16)
```

```
d.resize(3,3)
d
```

```
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
```

항목	`reshape()`	`resize()`
✅ 기본 동작	**새로운 배열**을 반환 (원본은 그대로)	**원본 배열 자체**의 shape을 변경
🧠 메모리	복사 또는 뷰(view)를 반환 (상황에 따라 다름)	in-place 연산 (기존 메모리 사용)
📎 원본 배열 변경 여부	❌ 원본 배열 변경 X	✅ 원본 배열이 직접 변경됨
🔍 요소 개수 불일치 시	❌ 오류 발생	✅ 부족하면 0 채움, 넘치면 잘림
⚠️ 레퍼런스 확인(refcheck)	없음	기본적으로 `refcheck=True`로 안전성 확인
💻 사용 예	데이터 구조 변형 시 (ex. 2D → 1D)	배열 자체를 확장/축소하고자 할 때

- 격자를 사용하여 확장 처리하기:
  - 1차원 배열 두개를 meshgrid 함수를 인자로 전달하면,
    첫번째 배열은 수직으로 확장, 두번째 배열은 수평적으로 확장
    → 두 배열의 결과는 동일한 형상을 갖는 2차원 배열로 만들어짐.

```
x = np.arange(3)
y = np.array([4,5,6,7])
```

```
print(x)
print(y)
```

```
[0 1 2]
[4 5 6 7]
```

```
x_, y_ = np.meshgrid(x,y)
```

```
x_
```

```
array([[0, 1, 2],
       [0, 1, 2],
       [0, 1, 2],
       [0, 1, 2]])
```

```
y_
```

```
array([[4, 4, 4],
       [5, 5, 5],
       [6, 6, 6],
       [7, 7, 7]])
```

- 위 결과를 concatenate 함수를 사용하여 처리할 수 있음.

```
x = np.arange(3)
y = np.array([4,5,6,7])
```

```
print(x)
print(y)
```

```
[0 1 2]
[4 5 6 7]
```

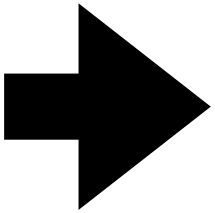
```
x_, y_ = np.meshgrid(x,y)
```

```
x_
```

```
array([[0, 1, 2],
       [0, 1, 2],
       [0, 1, 2],
       [0, 1, 2]])
```

```
y_
```

```
array([[4, 4, 4],
       [5, 5, 5],
       [6, 6, 6],
       [7, 7, 7]])
```



```
x_1 = x.reshape(1,-1)
x_1
```

```
array([[0, 1, 2]])
```

```
np.concatenate([x_1, x_1, x_1, x_1])
```

```
array([[0, 1, 2],
       [0, 1, 2],
       [0, 1, 2],
       [0, 1, 2]])
```

```
y_1 = y.reshape(-1,1)
y_1
```

```
array([[4],
       [5],
       [6],
       [7]])
```

```
np.concatenate([y_1,y_1,y_1], axis =1)
```

```
array([[4, 4, 4],
       [5, 5, 5],
       [6, 6, 6],
       [7, 7, 7]])
```

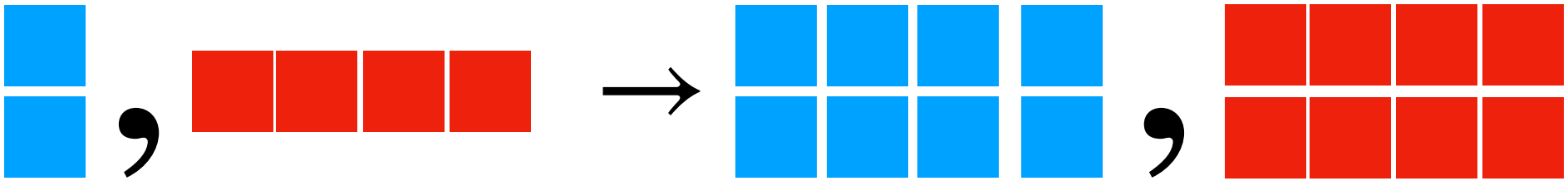
- 두 개의 array를 **broadcast** 를 사용하여 2차원으로 확장

```
A= np.arange(10,30,10)
B= np.arange(1,5)
print(A)
print(B)
```

```
[10 20]
[1 2 3 4]
```

```
np.broadcast_arrays(A.reshape(2,1), B.reshape(1,4))
```

```
(array([[10, 10, 10, 10],
        [20, 20, 20, 20]]),
 array([[1, 2, 3, 4],
        [1, 2, 3, 4]]))
```



- 격자 객체를 사용한 확장 처리 하기:
  - mgrid 클래스에 색인연산을 통해, 두 개의 객체가 생성되도록 슬라이스를 지정하여 실행하면, 두 개의 배열이 생성됨.

```
type(np.mgrid)
```

```
numpy.lib._index_tricks_impl.MGridClass
```

```
a, b = np.mgrid[10:30:10, 1:5]
```

```
a
```

```
array([[10, 10, 10, 10],
       [20, 20, 20, 20]])
```

```
b
```

```
array([[1, 2, 3, 4],
       [1, 2, 3, 4]])
```

=

```
s1 = slice(10,30,10)
s2 = slice(1,5)
type(s1)
```

```
slice
```

```
a_, b_ = np.mgrid[s1,s2]
```

```
a_
```

```
array([[10, 10, 10, 10],
       [20, 20, 20, 20]])
```

```
b_
```

```
array([[1, 2, 3, 4],
       [1, 2, 3, 4]])
```

- 실제 slice 클래스 (슬라이스 연산을 객체로 표현하는 클래스) 를 사용하여, mgrid에 전달하는 것과 동일함.

- 격자 객체를 사용한 확장 처리 하기:
  - 축만 추가하여 차원을 변경하는 기능 수행하는 함수: ogrid
  - mgrid = ogrid + broadcast_arrays

```
type(np.ogrid)
```

```
numpy.lib._index_tricks_impl.OGridClass
```

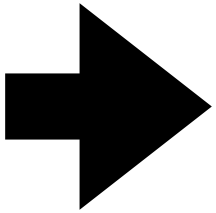
```
c, d = np.ogrid[s1,s2]
```

```
c
```

```
array([[10],
       [20]])
```

```
d
```

```
array([[1, 2, 3, 4]])
```



```
e, f = np.broadcast_arrays(c,d)
```

```
e
```

```
array([[10, 10, 10, 10],
       [20, 20, 20, 20]])
```

```
f
```

```
array([[1, 2, 3, 4],
       [1, 2, 3, 4]])
```

- 특정 조건만 추출하기

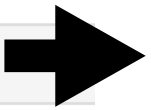
```
np.compress(condition, array, axis=None)
```

- condition: bool (0 또는 1) 배열
- axis 선택. 만약 axis=None (default)인 경우, flatten 시킨 후 조건 적용

```
a = np.array([[1,2],[3,4],[5,6]])
a
```

```
array([[1, 2],
       [3, 4],
       [5, 6]])
```

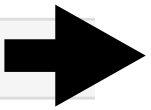
```
np.compress([0,1],a)
```



a.flatten() → [1, 2, 3, 4, 5, 6]

```
array([2])
```

```
np.compress([0,1,1,0,0,0],a)
```



a.flatten() → [1, 2, 3, 4, 5, 6]

```
array([2, 3])
```

```
np.compress([1, 0, 1], a, axis=0)
# → 행 0, 행 2 선택 → [[1, 2], [5, 6]]
```

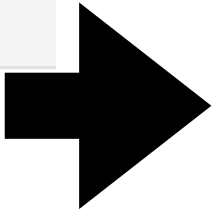
```
array([[1, 2],
       [5, 6]])
```

```
np.compress([0, 1], a, axis=1)
# → 열 1만 선택 → [[2], [4], [6]]
```

```
array([[2],
       [4],
       [6]])
```

```
a = np.array([[1,2],[3,4],[5,6]])
a
```

```
array([[1, 2],
       [3, 4],
       [5, 6]])
```



원소 중, 3으로 나누어서 나머지가 0이 아닌 원소

```
condition = np.mod(a, 3)
condition
```

```
array([[1, 2],
       [0, 1],
       [2, 0]])
```

```
np.compress(condition.flatten(),a)
```

```
array([1, 2, 4, 5])
```

```
np.extract(condition,a)
```

```
array([1, 2, 4, 5])
```

항목	`np.compress`	`np.where`	Boolean Indexing	`np.extract`
🎯 목적	축(`axis`) 기준 조건 선택	조건 인덱스 반환 또는 삼항 선택	조건을 만족하는 값 필터링	조건을 만족하는 값 필터링 (함수형)
🔢 조건 배열 형식	1D bool or 0/1 배열 (축 길이와 같아야 함)	입력과 같은 shape의 bool 배열	입력과 같은 shape의 bool 배열	입력과 같은 shape의 bool 배열
📐 축 지정 가능 여부	✅ 가능 (`axis=0` , `1` , `None`)	❌ 없음	❌ 없음	❌ 없음
↩️ 반환값 형식	조건에 맞는 행/열/원소 (shape 유지 또는 축소)	- `np.where(cond)` : 인덱스 튜플 - `np.where(cond, x, y)` : 조건 분기 결과	조건 만족 원소만 반환	조건 만족 원소 (항상 1D)
📊 반환 배열 차원	원본 shape 유지 or 줄어듦	조건 인덱스 (튜플) 또는 원본 shape 유지	보통 shape 축소됨	항상 1차원 (flatten)
🧠 특징	행/열 기준 조건 선택에 유리	위치 기반 조건 추출 및 조건 분기	Pythonic한 표현으로 직관적 사용	Boolean Indexing의 함수 버전
✅ 권장 사용 상황	축을 기준으로 선택해야 할 때	조건 위치가 필요하거나 if-else 처리 시	일반적 필터링에 가장 많이 사용	간결한 함수형 조건 선택

```
np.compress(condition.flatten(),a)
```

```
array([1, 2, 4, 5])
```

```
a[np.where(np.mod(a,3)>0)]
```

```
array([1, 2, 4, 5])
```

:

```
a[ np.mod(a,3) >0]
```

:

```
array([1, 2, 4, 5])
```

```
np.extract(condition,a)
```

```
array([1, 2, 4, 5])
```